Fresh: probador virtual de barbería con computación gráfica

Resumen ejecutivo del proyecto final de CS4016

Kalos Lazo Mera Sofía Herrera Gianpier Segovia
Universidad de Ingeniería y Tecnología (UTEC), Lima, Perú

Noviembre de 2026

Resumen

Fresh es una aplicación web para probarse barbas, tintes y cortes de pelo sobre la propia foto o con la cámara, antes de ir a la barbería. En lugar de generar una cara nueva con un modelo generativo, mide la geometría de la cara y del pelo de la persona y compone cada estilo sobre la foto original. Los modelos preentrenados solo ubican la cara, el pelo y la mano; todo lo demás es implementación propia: una triangulación de Delaunay con el algoritmo de Fortune, un *warp* afín por triángulo, un tinte en CIELAB, un motor de ocho cortes con relleno *pull-push*, física de la barba con Verlet, gaussianas 2D ajustadas con gradientes analíticos y un visor de *Gaussian Splatting* 3D en WebGL2. El sistema corre en CPU, procesa una foto en 0,1 a 2,5 s y se reproduce con un solo comando de Docker.

Palabras clave: triangulación de Delaunay, *warp* afín, CIELAB, *inpainting*, Verlet, *Gaussian Splatting*.

1. Introducción

Cortarse el pelo, dejarse la barba o teñirse son decisiones que se toman a ciegas: el resultado se ve cuando ya está hecho. Las aplicaciones que existen pegan una imagen encima de la foto, sin seguir la forma de la cara ni la luz, o usan modelos generativos que inventan la cara completa, pierden la identidad y necesitan GPU.

Fresh aborda el problema con computación gráfica: modela la cara y el pelo de la persona y compone sobre la foto original las capas necesarias (barba, color, pelo corto y piel). Así el resultado conserva la identidad, la iluminación y la textura. Las contribuciones son:

- un pipeline en el que los modelos solo ubican y el código del equipo hace toda la geometría y el render (unas 4 000 líneas propias frente a 600 que llaman a modelos);
- un motor de cortes que quita el pelo, reconstruye el fondo y hace crecer pelo corto con la luz de la foto;
- un tinte en CIELAB que conserva los mechones y puede teñir también la barba;
- física de la barba y un juego de afeitado en vivo;
- gaussianas 2D y 3D implementadas desde cero, con un visor propio en WebGL2.

2. Visión general

La figura 1 muestra el recorrido de una foto. MediaPipe [1, 2] entrega 478 puntos de la cara, que se convierten a los 68 puntos estándar, y un segmentador marca pelo, piel y fondo. Para el tinte se puede usar en su lugar un YOLOv8n-seg [3] que el equipo ajustó con Figaro1k [4] y CelebAMask-HQ [5] (mAP50 de máscara de 0,968).

Con los puntos se arma una malla de Delaunay y un *marco de la cabeza*: coordenadas (x, y) medidas en anchos de cabeza, con origen entre las orejas. Así un mismo parámetro vale para una foto de 600 px y para una de 4000 px. Sobre esa geometría, cada herramienta aplica su algoritmo y devuelve una imagen compuesta o, en el 3D, gaussianas que dibuja el navegador.

El servidor es un único proceso de FastAPI dentro de un contenedor Docker. El modo en vivo y el juego guardan estado entre cuadros, y cada llamada al pipeline corre en un hilo detrás de un candado. La interfaz es HTML y JavaScript sin compilación.

3. Métodos

3.1. Malla de Delaunay con Fortune

La triangulación de Delaunay [6] evita triángulos finos, que estirarían la textura. Se construye con el barrido de Fortune [7]: cada evento de círculo es un vértice de Voronoi y aporta un triángulo cuyo círculo circunscrito está vacío. Nuestra *beach line* es una lista enlazada, así que la implementación es $O(n^2)$ en el peor caso, en vez del $O(n \log n)$ que se logra con un árbol; con 83 puntos tarda 3 ms. Contra Bowyer–Watson [8, 9], Fortune encuentra además un triángulo válido del borde de la mandíbula que el otro método pierde por su supertriángulo finito. Para los casos degenerados (puntos cocirculares, como en una grilla) se perturban los puntos en una millonésima del tamaño de la nube y se descartan los triángulos sin área.

3.2. Barba

Una barba real de CelebAMask-HQ se deforma a la cara con un *warp* afín por triángulo: para cada píxel del triángulo destino se resuelve la afín inversa, se prueba si está dentro con coordenadas baricéntricas y se lee el origen con interpolación bilineal propia. La barba se

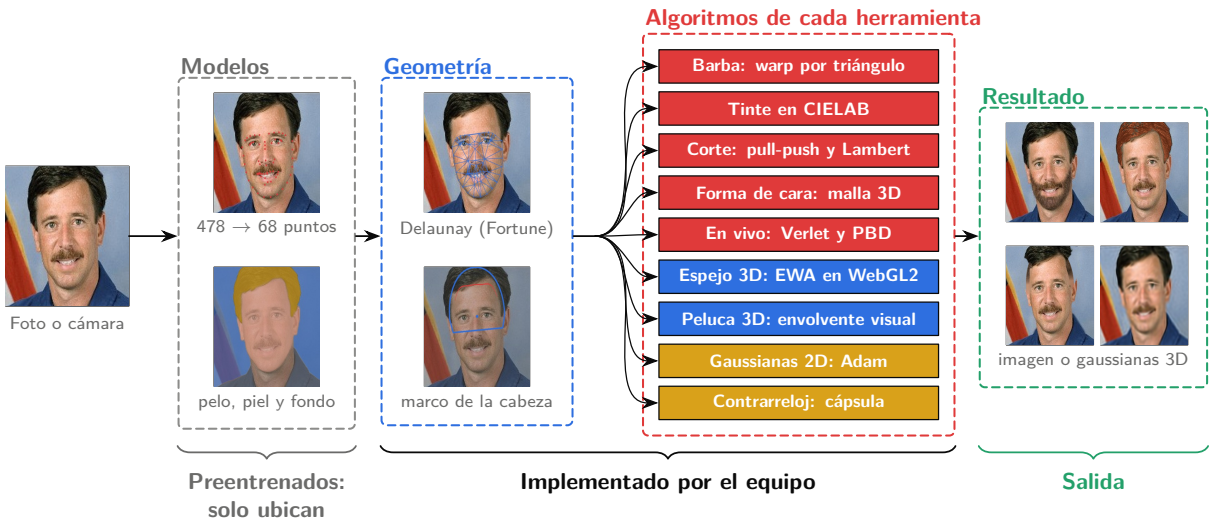


Figura 1: Pipeline de Fresh. En gris, los modelos preentrenados, que solo ubican; el resto lo implementó el equipo. Los colores de las herramientas siguen los grupos de la interfaz: Estilos (rojo), 3D (azul) y Juegos (amarillo).

compone en tres capas: *multiply*, $C = (1-\alpha)\,b + \alpha\,b\,m$, que oscurece sin tapar la luz de la piel; hebras con curvas de Bézier; y la alta frecuencia de la plantilla, que aporta los pelos reales sin su brillo.

3.3. Tinte en CIELAB

En CIELAB [10] la luminosidad está separada del color. La luminosidad del pelo se divide en sombreado $s = G_\sigma(L) - \bar{L}$ y detalle $d = L - G_\sigma(L)$, y la nueva luminosidad es

$$L' = 0{,}72\,L_t + 0{,}28\,\bar{L} + k\,(0{,}8\,s + 1{,}2\,d), \qquad (1)$$

con $k = \mathrm{clip}(20/s_L,\ 0{,}9,\ 1{,}8)$ para normalizar el contraste. La base va hacia el tinte y el detalle se conserva, así un pelo negro puede quedar rubio. El croma baja en las sombras, como en un pigmento real.

3.4. Motor de cortes

Cortar el pelo exige quitarlo y reconstruir lo que había detrás (figura 2). El *inpainting* de Telea [11] toma color de todo el borde del hueco, que incluye pelo, y deja "pelo fantasma". Usamos *pull-push* [12]: una pirámide en la que solo cuentan como conocidos los píxeles de fondo seguro. El cráneo es una superelipse ajustada a la coronilla de cada persona,

$$\left|\frac{x}{0{,}55}\right|^{2{,}15} + \left|\frac{\min(y,0)}{y_{\text{tope}}}\right|^{2{,}15} \le 1, \qquad (2)$$

y la línea del pelo sale de la frente visible o, con flequillo, de proporciones típicas [13]. El pelo corto toma el color propio y se sombrea con Lambert [14], con la luz estimada de la asimetría de brillo de la cara. Los ocho cortes son tablas de parámetros sobre el mismo marco.

3.5. Física de la barba y juego

En el modo en vivo, cada hebra es una cadena de cuatro partículas con la raíz anclada a la piel por coordenadas baricéntricas. Se integra con Verlet [15] amortiguado, $x \leftarrow x + 0{,}86\,(x - x_{\text{prev}}) + g\,h^2$, y restricciones de distancia (PBD) [16, 17] que mantienen el largo. Cada tramo se dibuja como una gaussiana 2D y se acumula con $\alpha = 1 - e^{-A}$. En el juego Contrarreloj, el dedo índice corta las hebras cercanas al segmento recorrido entre dos cuadros (una cápsula), porque a 10 cuadros por segundo el dedo salta.

3.6. Gaussianas 2D y 3D

Un retrato se reconstruye como suma de N gaussianas 2D, al estilo de GaussianImage [18]. Cada una guarda el factor de Cholesky de la inversa de su covarianza, $\Sigma^{-1} = LL^\top$, para que siempre sea válida, y se optimiza con gradientes analíticos y Adam [19] escritos a mano, sin PyTorch.

El espejo 3D construye, desde una foto y la malla 3D de MediaPipe, más de 100 000 gaussianas en forma de discos tangentes, y las dibuja con un visor propio en WebGL2. El *vertex shader* proyecta cada covarianza con EWA [20],

$$\Sigma_{2D} = J\,W\,\Sigma\,W^\top J^\top + 0{,}3\,I, \qquad (3)$$

un *counting sort* las ordena de atrás hacia adelante y el *fragment shader* las mezcla, como en 3D Gaussian Splatting [21]. La peluca 3D, en cambio, escanea el peinado con la cámara y obtiene su forma con una envolvente visual [22] calculada rebanada por rebanada.

4. Resultados

La figura 3 muestra los ocho cortes sobre tres personas con pelo distinto: en todos el fondo queda limpio y la cara no cambia. La tabla 1 reúne las cifras principales. Además de los retratos de la NASA, los cortes y el tinte se revisaron sobre 26 caras distintas, y 54 pruebas automáticas corren en GitHub Actions en cada cambio.

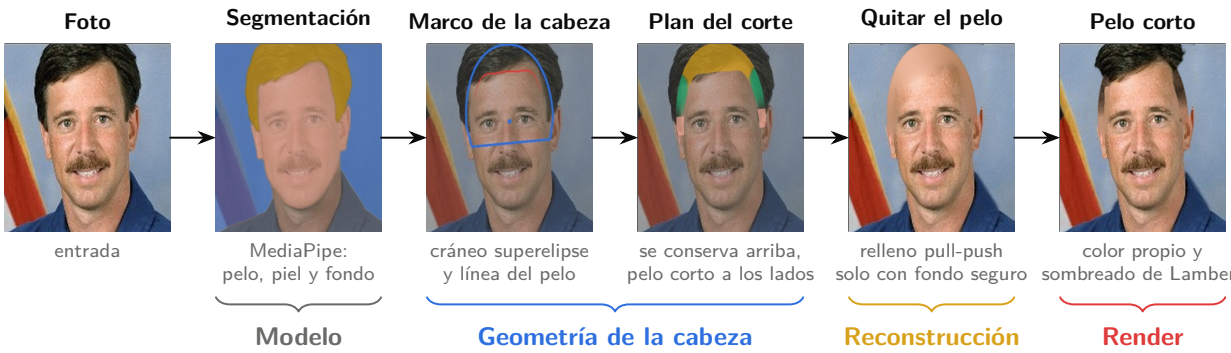


Figura 2: Motor de cortes sobre un retrato de dominio público de la NASA (degradado medio): segmentación, marco de la cabeza con el cráneo y la línea del pelo, plan del corte, relleno con *pull-push* y pelo corto sombreado.

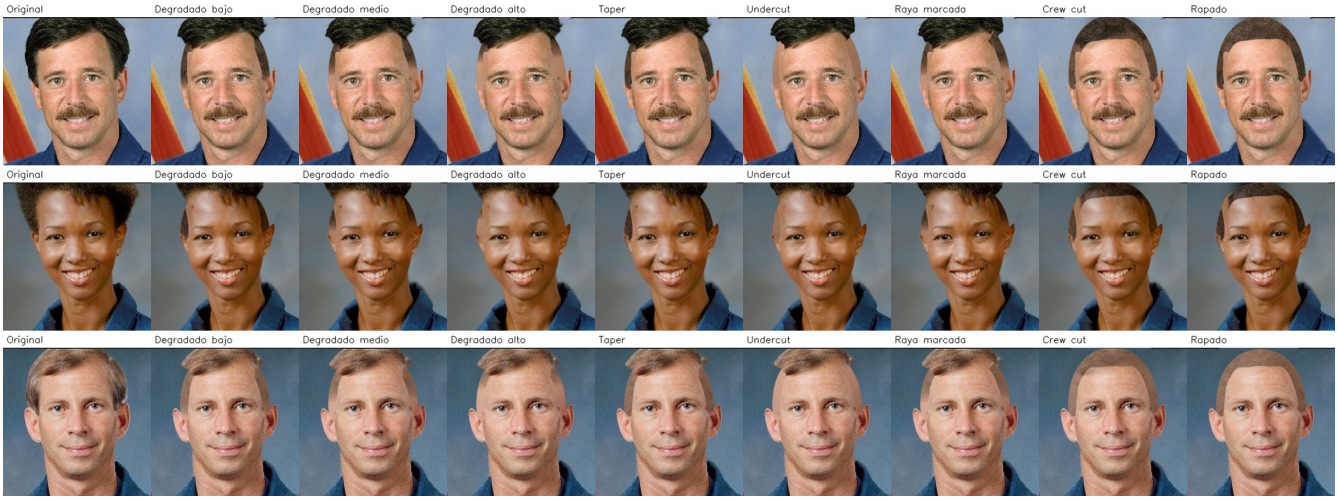


Figura 3: Los ocho cortes sobre tres retratos de dominio público: degradado bajo, medio y alto, *taper*, *undercut*, raya marcada, *crew cut* y rapado.

Medida	Valor
Corte, foto de 1024–1638 px (CPU)	0,6 – 1,9 s
Corte, foto de 4000 px	0,8 – 1,1 s
Tinte con YOLO y barba	2,5 s
Espejo 3D (105 000 gaussianas)	2,6 s
Barba en vivo, por cuadro	61 ms
Física en vivo, por cuadro	72 ms
YOLO de pelo: mAP50 de máscara	0,968
IoU entre YOLO y MediaPipe	0,82 – 0,94
Gaussianas 2D: PSNR con 20 y 2000	18,4 y 34,5 dB
Delaunay con 83 puntos	3 ms

Tabla 1: Tiempos y calidad. El PSNR sube unos 8 dB cada vez que se multiplica por 10 el número de gaussianas.

5. Limitaciones

- Si una mano pasa delante de la cara en vivo, la barba se dibuja encima.
- Si un flequillo largo tapa un ojo, el rapado deja mechones, porque nunca se modifica la cara.
- El espejo 3D usa una sola vista: no tiene nuca y el relieve se estira con giros grandes.
- El modo en vivo corre a unos 10 cuadros por segundo, porque cada cuadro se procesa en el servidor.

6. Conclusiones

Fresh muestra que se puede probar un estilo de barbería de forma creíble sin modelos generativos: basta con ubicar la cara y el pelo, y resolver el resto con geometría, color, relleno de imágenes, física y *splatting*. Cada pieza es explicable y corre en CPU. Como trabajo futuro quedan el tinte en vivo con un *shader* de WebGL y un 3D Gaussian Splatting entrenado con varias vistas para completar la cabeza.

Aportes. Kalos Lazo: arquitectura, tinte, YOLO, motor de cortes, forma de cara, interfaz y despliegue. Gianpier Segovia: barba, Delaunay de Fortune, *warp*, física, gaussianas 2D, juego y espejo 3D. Sofía Herrera: peluca 3D y laboratorio.

Disponibilidad. El código, el informe completo con todas las fórmulas y la guía de uso están en el repositorio del proyecto. La aplicación se levanta con `docker compose up -d -build app`.

Referencias

[1] Y. Kartynnik, A. Ablavatski, I. Grishchenko, and M. Grundmann, “Real-time facial surface geometry from

monocular video on mobile GPUs." arXiv:1907.06724, 2019.

[2] C. Lugaresi, J. Tang, H. Nash, C. McClanahan, E. Uboweja, M. Hays, F. Zhang, C.-L. Chang, M. G. Yong, J. Lee, W.-T. Chang, W. Hua, M. Georg, and M. Grundmann, "MediaPipe: A framework for building perception pipelines." arXiv:1906.08172, 2019.

[3] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8." Software, versión 8, 2023. Disponible en: https://github.com/ultralytics/ultralytics.

[4] M. Svanera, U. R. Muhammad, R. Leonardi, and S. Benini, "Figaro, hair detection and segmentation in the wild," in *Proc. IEEE Int. Conf. Image Processing (ICIP)*, pp. 933–937, 2016.

[5] C.-H. Lee, Z. Liu, L. Wu, and P. Luo, "MaskGAN: Towards diverse and interactive facial image manipulation," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, pp. 5549–5558, 2020.

[6] B. Delaunay, "Sur la sphère vide," *Bulletin de l'Académie des Sciences de l'URSS, Classe des Sciences Mathématiques et Naturelles*, no. 6, pp. 793–800, 1934.

[7] S. Fortune, "A sweepline algorithm for Voronoi diagrams," *Algorithmica*, vol. 2, no. 1–4, pp. 153–174, 1987.

[8] A. Bowyer, "Computing Dirichlet tessellations," *The Computer Journal*, vol. 24, no. 2, pp. 162–166, 1981.

[9] D. F. Watson, "Computing the n-dimensional Delaunay tessellation with application to Voronoi polytopes," *The Computer Journal*, vol. 24, no. 2, pp. 167–172, 1981.

[10] Commission Internationale de l'Éclairage, "Colorimetry," Tech. Rep. Publication 15:2004, CIE, Vienna, 2004.

[11] A. Telea, "An image inpainting technique based on the fast marching method," *Journal of Graphics Tools*, vol. 9, no. 1, pp. 23–34, 2004.

[12] S. J. Gortler, R. Grzeszczuk, R. Szeliski, and M. F. Cohen, "The lumigraph," in *Proc. ACM SIGGRAPH*, pp. 43–54, 1996.

[13] L. G. Farkas, *Anthropometry of the Head and Face*. New York: Raven Press, 2nd ed., 1994.

[14] S. Marschner and P. Shirley, *Fundamentals of Computer Graphics*. Boca Raton, FL: CRC Press, 4th ed., 2016.

[15] L. Verlet, "Computer "experiments" on classical fluids. I. Thermodynamical properties of Lennard-Jones molecules," *Physical Review*, vol. 159, no. 1, pp. 98–103, 1967.

[16] T. Jakobsen, "Advanced character physics," in *Proc. Game Developers Conference*, (San José, CA), 2001.

[17] M. Müller, B. Heidelberger, M. Hennix, and J. Ratcliff, "Position based dynamics," *Journal of Visual Communication and Image Representation*, vol. 18, no. 2, pp. 109–118, 2007.

[18] X. Zhang, X. Ge, T. Xu, D. He, Y. Wang, H. Qin, G. Lu, J. Geng, and J. Zhang, "GaussianImage: 1000 FPS image representation and compression by 2D Gaussian splatting," in *Proc. European Conf. Computer Vision (ECCV)*, 2024.

[19] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2015.

[20] M. Zwicker, H. Pfister, J. van Baar, and M. Gross, "EWA volume splatting," in *Proc. IEEE Visualization*, pp. 29–36, 2001.

[21] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3D Gaussian splatting for real-time radiance field rendering," *ACM Trans. Graphics*, vol. 42, no. 4, pp. 139:1–139:14, 2023.

[22] A. Laurentini, "The visual hull concept for silhouette-based image understanding," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 16, no. 2, pp. 150–162, 1994.